InvestoraAI

AI-Powered Stock Market Intelligence Platform

VERSION	DATE	STATUS
1.0.0	March 2026	✅ Production

FRONTEND https://investora-ai.vercel.app

BACKEND https://investoraai.onrender.com

Table of Contents

1 Project Overview

2 Technology Stack

3 System Architecture

4 LangGraph Agent — Nodes & Data Flow

5 Backend API Reference

6 Frontend Structure

7 External APIs & Integrations

8 Environment Variables — Full Reference

9 Deployment — Render (Backend)

10 Deployment — Vercel (Frontend)

11 Local Development Setup

12 n8n Webhook Workflows

13 Known Limitations & Future Roadmap

1 Project Overview

InvestoraAI is a full-stack AI-powered financial intelligence platform that runs a weekly automated stock market analysis using a LangGraph ReAct agent. It pulls real-time market prices, fundamental data, and financial news — computes quality + momentum scores for a configurable stock universe — and delivers a structured weekly report to a React dashboard.

What It Does

📊 Weekly Analysis Engine

A LangGraph ReAct agent iterates through a stock universe (default: 10 tickers), fetches market + fundamental + news data, computes composite scores, and synthesizes evidence into actionable intelligence.

🤖 AI Synthesis

GPT-4o-mini synthesizes raw signal data into concise company insights, identifies top opportunities, and classifies stocks by quality (Strong/Good/Solid/Weak) and momentum (Strong/Positive/Neutral/Negative).

📡 Live Dashboard

A React frontend displays top-ranked opportunities, strategy breakdowns, performance metrics, run history, and per-ticker history — with live SSE streaming of the agent's progress.

🔔 Alert System

High-signal stocks (score ≥ 8.5 or ≤ 1.5) trigger alerts via n8n webhooks. Weekly candidates (score 7.0–8.4) are posted separately. All results can be forwarded to Telegram.

Stock Universe (Default)

The default universe covers 10 large-cap US equities across technology, finance, and consumer sectors:

`AAPL · AMZN · GOOGL · JPM · MA · META · MSFT · NVDA · TSLA · V`

Scoring Model

COMPONENT	WEIGHT	INPUTS	OUTPUT
Quality Score	`0.55`	P/E ratio, earnings growth, revenue trend, balance sheet	0–10 → Strong / Good / Solid / Weak

COMPONENT	WEIGHT	INPUTS	OUTPUT
Momentum Score	`0.45`	Price change, RSI proxy, volume, news sentiment	0–10 → Strong / Positive / Neutral / Negative
Composite Score	—	quality × 0.55 + momentum × 0.45	0–10 final ranking

2 Technology Stack

Frontend

React	Vite	TypeScript	Tailwind CSS
v18.3	v6.3	v5.8	v3.4
shadcn/ui	**React Query**	**React Router**	**Recharts**
Radix-based	v5.83	v6.30	v2.15
Framer Motion	**Sentry**	**Vitest**	**Vercel**
v12.34	v10.41	v3.2	Deploy

Backend

Python	FastAPI	LangGraph	LangChain
3.12.0	latest	ReAct Agent	Tools
OpenAI	**Uvicorn**	**SQLite**	**Render**
GPT-4o-mini	ASGI Server	Event Store	Deploy

External Services

SERVICE	PROVIDER	PURPOSE	FREE TIER
Market Data	Marketstack	Stock prices, OHLCV, historical data	100 req/month
Fundamentals	Financial Modeling Prep	P/E, earnings, balance sheet	250 req/day

SERVICE	PROVIDER	PURPOSE	FREE TIER
News	Finnhub	News articles, sentiment	60 req/min
AI/LLM	OpenAI	ReAct planning + synthesis	Pay-per-token
Vector DB	Pinecone	RAG memory / memos (optional)	Free tier
Automation	n8n (self-hosted)	Webhooks, Telegram alerts	Self-hosted
Error Tracking	Sentry	Frontend error monitoring	Free tier

3 System Architecture

High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                          USER / BROWSER                          │
│                https://investora-ai.vercel.app                   │
└──────────────────────────────────────────────────────────────────┘
                            │  HTTPS requests
                            ▼
┌──────────────────────────────────────────────────────────────────┐
│                     VERCEL (Frontend CDN)                        │
│   React 18 + Vite 6 + TypeScript + Tailwind + shadcn/ui          │
│                                                                  │
│   Pages: Dashboard · Run Analysis · History · Ticker Detail      │
│   Lib:   report.ts  (React Query hooks + SSE streaming)          │
└──────────────────────────────────────────────────────────────────┘
                            │  REST + SSE
                            ▼
┌──────────────────────────────────────────────────────────────────┐
│                   RENDER (Backend Cloud)                         │
│             https://investoraai.onrender.com                     │
│                                                                  │
│   FastAPI  ──►  LangGraph ReAct Agent                            │
│                     │                                            │
│               ┌─────┴─────┐                                      │
│               ▼           ▼                                      │
│        MCP Tools        Scoring Engine                           │
│     Market | FMP | News   Quality × 0.55 + Momentum × 0.45       │
│               │                                                  │
│         SQLite DB         data/reports/*.json                    │
│         (event store)     (served by /latest-report)             │
└──────────────────────────────────────────────────────────────────┘
      │           │
      ▼           ▼
┌──────────┐  ┌──────────────────────────────┐
│ OpenAI   │  │   External Data APIs         │
│ GPT-4o   │  │  Marketstack · FMP · Finnhub │
│  mini    │  │  Pinecone (optional RAG)     │
└──────────┘  └──────────────────────────────┘
      │
      ▼
┌──────────────────────────────┐
│  n8n (local Docker :5678)    │
│  Telegram alerts             │
│  Webhook automations         │
└──────────────────────────────┘
```

Repository Structure

```
InvestoraAI/
├── src/                          # React frontend source
│   ├── components/              # Reusable UI components
│   ├── pages/                   # Route-level pages (Index, NotFound)
│   ├── lib/
│   │   └── report.ts           # API hooks, TypeScript interfaces, SSE
│   └── App.tsx                  # Router setup
│
├── langgraph/                    # Python backend
│   ├── app/
│   │   ├── api.py              # FastAPI app + CORS + endpoints
│   │   ├── graph.py            # LangGraph agent definition
│   │   ├── state.py            # AgentState TypedDict
│   │   ├── scoring.py          # Quality + Momentum scoring logic
│   │   ├── reporting.py        # Report assembly + JSON serialization
│   │   ├── run_weekly.py       # Entry point: run_weekly() / run_analysis()
│   │   ├── event_store.py      # SQLite run history
│   │   ├── n8n_client.py       # Webhook posting logic
│   │   ├── alert_client.py     # Alert signal posting
│   │   ├── monitor_client.py   # Monitor webhook posting
│   │   └── mcp_tools/
│   │       ├── market_data_tool.py   # Marketstack wrapper
│   │       ├── fundamentals_tool.py  # FMP wrapper
│   │       ├── news_tool.py          # Finnhub wrapper
│   │       ├── rag_tool.py           # Pinecone RAG (optional)
│   │       └── mock_tools.py         # Mock data for testing
│   ├── data/
│   │   ├── reports/            # Generated JSON reports (YYYY-MM-DD.json)
│   │   ├── cache/              # Hashed API response cache
│   │   └── investora.db        # SQLite event store
│   ├── .env                    # Local environment variables (gitignored)
│   ├── .python-version         # Pins Python 3.12.0 for Render
│   └── requirements.txt        # Python dependencies
│
├── vercel.json                   # Vercel SPA config + rewrites
├── render.yaml                   # Render service config
└── package.json                  # Node deps (engines: node 20.x)
```

4 LangGraph Agent — Nodes & Data Flow

The core analysis engine is a **LangGraph ReAct agent** that runs as a directed graph. Each node performs a discrete step; the graph streams `node_complete` events back to the frontend via SSE (Server-Sent Events).

Graph Execution Flow

```
init_state
     │
     ▼
plan_next_action  ◀──────────────────────────────┐
     │                                           │
     ▼                                           │
execute_tool_action  (calls Market / FMP / News / RAG)   │
     │                                           │
     └──────────────────────────────────────────▶┘

     │  (when all tickers processed)
     ▼
compute_scores  (Quality × 0.55 + Momentum × 0.45)
     │
     ▼
synthesize_evidence  (GPT-4o-mini → insights, rankings)
     │
     ▼
emit_signals  (classify ALERT / WEEKLY_CANDIDATE / MONITOR)
     │
     ├──▶ post_alerts      (score ≥ 8.5 or ≤ 1.5)
     ├──▶ post_candidates  (score 7.0 - 8.4)
     │
     ▼
assemble_report_json  (build ReportPayload TypedDict)
     │
     ▼
assemble_markdown  (human-readable version)
     │
     ▼
post_to_n8n  (full report → n8n webhook)
     │
     ▼
persist_report  (write data/reports/YYYY-MM-DD.json)
     │
```

```
    ▼
persist_snapshot  (write SQLite run record + monitor webhook)
    |
    ▼
 DONE  →  SSE: {"type": "done", "run_id": "..."}
```

Node Descriptions

`init_state`

Initialises the AgentState with run_id, date, ticker list, and configuration flags.

`plan_next_action`

ReAct planner decides which tool to call next for which ticker (or terminates the tool loop).

`execute_tool_action`

Executes the chosen MCP tool (market_data, fundamentals, news, or rag). Caches results to disk.

`compute_scores`

Calculates quality score (0–10) and momentum score (0–10) for each ticker and combines them.

`synthesize_evidence`

Sends scored data to GPT-4o-mini. Returns ranked opportunities, insights, and rating labels.

`emit_signals`

Classifies each ticker as ALERT_EVENT, WEEKLY_CANDIDATE, or MONITOR based on score thresholds.

`post_alerts / post_candidates`

Posts high-signal and candidate tickers to separate n8n webhook endpoints.

`assemble_report_json`

Builds the full `ReportPayload` TypedDict ready for the dashboard API.

`assemble_markdown`

Generates a human-readable Markdown version of the report.

`post_to_n8n`

POSTs the complete report payload to the main n8n webhook for downstream automation.

`persist_report`

Writes `data/reports/YYYY-MM-DD.json` — served by the `/latest-report` endpoint.

`persist_snapshot`

Records run metadata to SQLite (run_id, error_count, signal_count, failed_tickers).

SSE Streaming Protocol

The `/run-analysis-stream` endpoint runs the graph in a background thread and yields SSE events:

```
# node_complete — emitted after each graph node finishes
data: {"type": "node_complete", "node": "compute_scores", "run_id": "abc-123"}

# done — emitted when the entire graph finishes successfully
data: {"type": "done", "run_id": "abc-123"}

# error — emitted if the graph throws an exception
```

```
data: {"type": "error", "run_id": "abc-123", "message": "..."}

# ping — emitted every 300s to keep the connection alive
data: {"type": "ping"}
```

5 Backend API Reference

Base URL: `https://investoraai.onrender.com`

Endpoints

GET /latest-report

Returns the most recently generated weekly report JSON. Reads the lexicographically latest file from `data/reports/` .

FIELD	VALUE
Auth required	No
Response	`ReportPayload` JSON object
Error (no report)	`404 {"detail": "No reports found"}`
Frontend cache	1 hour (staleTime in React Query)

POST /run-analysis-stream

Starts a full LangGraph analysis run and streams progress events as SSE. This is the primary endpoint used by the dashboard "Run Analysis" button.

REQUEST BODY

```
{
  "tickers": ["AAPL", "MSFT"],  // optional — defaults to full universe
  "skip_synthesis": false,          // skip GPT synthesis step
  "no_post": false                 // skip n8n webhook posting
}
```

RESPONSE

Content-Type: `text/event-stream` — streams SSE events until completion.

POST /run-weekly 🔒 CRON_SECRET

Runs the full weekly analysis synchronously (blocking). Requires `Authorization: Bearer <CRON_SECRET>` header when `CRON_SECRET` is set.

```
{
  "run_date": "2026-03-04",  // optional ISO date override
  "no_post": false           // skip n8n webhook posting
}
```

GET /run-history?limit=10

Returns metadata for the N most recent analysis runs from SQLite.

FIELD	TYPE	DESCRIPTION
`run_id`	string	UUID of the run
`run_date`	string	ISO date (YYYY-MM-DD)
`timestamp`	string	Full ISO datetime
`scope`	string	"full" or "fast"
`error_count`	number	Number of tool errors
`failed_tickers`	string[]	Tickers that had errors
`signal_count`	number	Number of signals emitted

GET /ticker-history/{ticker}?weeks=6

Returns signal events for a specific ticker over the last N weeks (max 52) from SQLite.

ReportPayload Schema

```
{
  "report": {
    "title": "Weekly Market Intelligence — Mar 04 2026",
    "run_date": "2026-03-04",
    "performance": { "weekly_change_percent": 2.4, "since_date": "...", "trend":
  },
  "top_opportunities": [
    {
      "rank": 1, "score": 8.7, "prior_score": 8.2,
      "company": { "name": "NVIDIA", "ticker": "NVDA", "icon": "🟢" },
```

```
        "quality": { "rating": "Strong", "score": 9.1 },
        "momentum": { "rating": "Strong", "score": 8.2 },
        "weekly_change": { "value": 3.4, "direction": "up" }
      }
    ],
    "strategy_breakdown": { "quality": [...], "momentum": [...] },
    "insights": [{ "company": "NVDA", "text": "Strong earnings growth..." }],
    "data_sources": [{ "name": "Marketstack", "type": "market" }],
    "tool_errors": [{ "ticker": "TSLA", "tool": "market", "error": "API quota excee
    "system_metadata": { "vector_index": { "provider": "pinecone", ... } }
  }
```

6 Frontend Structure

Key Pages & Components

FILE	ROUTE	DESCRIPTION
`src/pages/Index.tsx`	`/`	Main dashboard — stats bar, top opportunities, strategy breakdown, news feed
`src/pages/NotFound.tsx`	`*`	404 page
`src/lib/report.ts`	—	All TypeScript interfaces, React Query hooks, SSE streaming hook
`src/App.tsx`	—	Router setup, QueryClient provider, theme provider

React Query Hooks (src/lib/report.ts)

HOOK	ENDPOINT	CACHE TTL
`useLatestReport()`	GET /latest-report	1 hour
`useRunHistory()`	GET /run-history?limit=10	5 minutes
`useRunWeekly()`	POST /run-weekly (mutation)	—
`useStreamRun()`	POST /run-analysis-stream (SSE)	—

Environment Variable (Frontend)

VARIABLE	VALUE	WHERE SET
`VITE_API_BASE_URL`	`https://investoraai.onrender.com`	Vercel → Settings → Environment Variables

⚠️ **Important:** `VITE_API_BASE_URL` is baked into the bundle at build time by Vite. Any change to this variable requires a full Vercel redeploy to take effect.

Build Configuration

```
// package.json — key settings
{
  "engines": { "node": "20.x" },      // pins Node version on Vercel
  "scripts": {
    "dev":    "vite",
    "build":  "vite build",
    "test":   "vitest run"
  }
}

// vercel.json — SPA routing fix
{
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "framework": "vite",
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

7 External APIs & Integrations

1. Marketstack — Market Data

DETAIL	VALUE
Dashboard	`https://marketstack.com/dashboard`
Base URL	`https://api.marketstack.com/v1`
Env var	`MARKET_DATA_API_KEY` + `MARKET_DATA_API_BASE_URL`
Free tier	100 requests / month
Used for	EOD prices, OHLCV data, weekly price change calculation
⚠️ Caution	With 10 tickers, each full run uses ~10–20 requests. Monitor usage carefully.

2. Financial Modeling Prep — Fundamentals

DETAIL	VALUE
Dashboard	`https://financialmodelingprep.com/developer/docs`
Base URL	`https://financialmodelingprep.com`
Env var	`FUNDAMENTALS_API_KEY` + `FUNDAMENTALS_API_BASE_URL`
Free tier	250 requests / day
Used for	P/E ratio, EPS, revenue growth, balance sheet health metrics

3. Finnhub — News & Sentiment

DETAIL	VALUE
Dashboard	`https://finnhub.io/dashboard`
Base URL	`https://finnhub.io/api/v1`

DETAIL	VALUE
Env var	`NEWS_API_KEY` + `NEWS_API_BASE_URL`
Free tier	60 requests / minute
Used for	Company news articles, sentiment scores, earnings calendars

4. OpenAI — AI Synthesis

DETAIL	VALUE
Dashboard	`https://platform.openai.com/api-keys`
Env vars	`OPENAI_API_KEY` , `OPENAI_MODEL` , `SYNTHESIS_MODEL`
Default model	`gpt-4o-mini` (both planner and synthesis)
Used for	ReAct planning loop decisions + evidence synthesis into insights
Token limits	`MAX_TOKENS=1000` , `OPENAI_TIMEOUT=30`

5. Pinecone — Vector Memory (Optional)

DETAIL	VALUE
Dashboard	`https://app.pinecone.io`
Env vars	`PINECONE_HOST` , `PINECONE_API_KEY`
Used for	RAG memos — stores historical analysis summaries for context enrichment
Optional	If not set, the RAG tool soft-fails and analysis continues without memory

8 Environment Variables — Full Reference

Backend (Render / langgraph/.env)

VARIABLE	REQUIRED	DEFAULT	DESCRIPTION
`OPENAI_API_KEY`	Required	—	OpenAI API key for ReAct planner + synthesis
`OPENAI_MODEL`	Optional	`gpt-4o-mini`	Model for ReAct planning loop
`SYNTHESIS_MODEL`	Optional	`gpt-4o-mini`	Model for evidence synthesis step
`MARKET_DATA_API_KEY`	Required*	—	Marketstack API key
`MARKET_DATA_API_BASE_URL`	Optional	`https://api.marketstack.com/v1`	Marketstack base URL
`FUNDAMENTALS_API_KEY`	Required*	—	Financial Modeling Prep (FMP) API key
`FUNDAMENTALS_API_BASE_URL`	Optional	`https://financialmodelingprep.com`	FMP base URL
`NEWS_API_KEY`	Required*	—	Finnhub API key
`NEWS_API_BASE_URL`	Optional	`https://finnhub.io/api/v1`	Finnhub base URL
`PINECONE_API_KEY`	Optional	—	Pinecone API key for RAG memory
`PINECONE_HOST`	Optional	—	Pinecone index host URL
`N8N_WEBHOOK_URL`	Optional	—	n8n webhook for full report payload
`ALERT_WEBHOOK_URL`	Optional	—	n8n webhook for ALERT_EVENT signals
`CANDIDATE_WEBHOOK_URL`	Optional	—	n8n webhook for WEEKLY_CANDIDATE signals

VARIABLE	REQUIRED	DEFAULT	DESCRIPTION
`MONITOR_WEBHOOK_URL`	Optional	—	n8n webhook for run complete/error monitoring
`API_SECRET_KEY`	Optional	—	General API secret key for internal security
`CRON_SECRET`	Optional	—	Bearer token for /run-weekly cron endpoint (disabled if blank)
`CORS_ORIGINS`	Required	—	Comma-separated allowed origins e.g. `https://investora-ai.vercel.app`
`USE_MOCK_DATA`	Optional	`false`	Set `true` to use mock data (no API keys needed)
`ENVIRONMENT`	Optional	`production`	Environment name for logging
`REDIS_URL`	Optional	—	Redis connection URL for caching (disabled if blank)
`SENTRY_DSN`	Optional	—	Sentry DSN for error tracking
`RATE_LIMIT`	Optional	`20/minute`	API rate limiting
`MAX_TOKENS`	Optional	`1000`	OpenAI max tokens per call
`OPENAI_TIMEOUT`	Optional	`30`	OpenAI request timeout in seconds
`GRAPH_RECURSION_LIMIT`	Optional	`120`	LangGraph recursion limit
`STOCK_UNIVERSE`	Optional	10 default tickers	Comma-separated tickers to analyse
`QUALITY_WEIGHT`	Optional	`0.55`	Weight for quality score in composite

VARIABLE	REQUIRED	DEFAULT	DESCRIPTION
`MOMENTUM_WEIGHT`	Optional	`0.45`	Weight for momentum score in composite

* Required unless `USE_MOCK_DATA=true`

Frontend (Vercel Environment Variables)

VARIABLE	REQUIRED	VALUE
`VITE_API_BASE_URL`	Required	`https://investoraai.onrender.com`

9 Deployment — Render (Backend)

✅ **Status: Live** — `https://investoraai.onrender.com`

render.yaml Configuration

```yaml
services:
  - type: web
    name: investora-backend
    runtime: python
    rootDir: langgraph
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn app.api:app --host 0.0.0.0 --port $PORT
    envVars:
      - key: PYTHON_VERSION
        value: 3.12.0
      - key: OPENAI_API_KEY
        sync: false       # must be set in Render dashboard
      - key: MARKET_DATA_API_KEY
        sync: false
      # ... etc
```

Python Version Pinning

A `langgraph/.python-version` file pins Python to `3.12.0`. This is critical — Render defaults to Python 3.14+ which is incompatible with several LangChain dependencies.

```
3.12.0
```

Manual Setup Steps (First Deploy)

1. **Connect GitHub repo** — Render → New Web Service → connect `monamohanty1987/InvestoraAI`
2. **Set Root Directory** → `langgraph`
3. **Set Build Command** → `pip install -r requirements.txt`
4. **Set Start Command** → `uvicorn app.api:app --host 0.0.0.0 --port $PORT`
5. **Add all secret environment variables** in Render dashboard (API keys, CORS_ORIGINS, etc.)

Deploy — Render auto-deploys on every push to the connected branch

6

Free Tier Limitations

⚠️ **Render Free Plan spins down after 15 minutes of inactivity.** The first request after sleep takes 30–60 seconds (cold start). Upgrade to Starter ($7/mo) for always-on service.

Persistent Storage Note

⚠️ Render's free plan has **ephemeral filesystem storage**. Files written to `data/reports/` and `data/investora.db` are lost on every deploy/restart. To persist data, either: (a) add a Render Disk ($0.25/GB/month), or (b) commit generated reports to git.

10 Deployment — Vercel (Frontend)

✅ **Status: Live** — `https://investora-ai.vercel.app`

Key Settings

SETTING	VALUE
Framework	Vite
Build Command	`npm run build`
Output Directory	`dist`
Node Version	`20.x` (pinned via `engines.node` in package.json)
Production Branch	`Investora-AI`
SPA Routing	All routes rewrite to `/index.html` via `vercel.json`

Important Fixes Applied

PROBLEM	FIX
Vercel used Bun instead of npm	Removed `bun.lockb` from git
Node v24 incompatible with Vite 5	Added `"engines": {"node": "20.x"}` to package.json
vitest@3 requires Vite 6	Upgraded to `vite@6.3.0` + `@vitejs/plugin-react-swc@4`
Production URL showed DEPLOYMENT_NOT_FOUND	Changed production branch from `main` to `Investora-AI`

Deployment Steps

1. **Connect GitHub repo** → Vercel → New Project → import `monamohanty1987/InvestoraAI`
2. **Set Environment Variable** → `VITE_API_BASE_URL = https://investoraai.onrender.com`
3. **Set Production Branch** → Settings → Environments → Production → Branch Tracking → `Investora-AI`

4. **Trigger first deployment** → push any commit to `Investora-AI` branch
5. **Verify** → open `https://investora-ai.vercel.app`

Auto-Deploy

Every push to the `Investora-AI` branch automatically triggers a Production deployment. Other branches create Preview deployments at temporary URLs.

To Redeploy Without Code Changes

```
cd C:\InvestoraAI
git commit --allow-empty -m "trigger: redeploy"
git push origin Investora-AI
```

11 Local Development Setup

Prerequisites

Node.js
v20.x (LTS)

Python
3.12.0 (via pyenv)

Docker
For n8n (optional)

Frontend Setup

```
# Navigate to project root
cd C:\InvestoraAI

# Install dependencies
npm install

# Create .env.local with backend URL
# VITE_API_BASE_URL=http://localhost:8000

# Start dev server (runs on http://localhost:8080)
npm run dev
```

Backend Setup

```
# Navigate to backend directory
cd C:\InvestoraAI\langgraph

# Create virtual environment
python -m venv venv
venv\Scripts\activate       # Windows
source venv/bin/activate    # Mac/Linux

# Install dependencies
pip install -r requirements.txt

# Copy and edit environment file
copy .env.example .env
# Edit .env with your API keys

# Start backend (runs on http://localhost:8000)
uvicorn app.api:app --reload --port 8000
```

Running Mock Mode (No API Keys)

```
# In langgraph/.env:
USE_MOCK_DATA=true

# Mock data covers these tickers with realistic variance:
# AAPL · MSFT · NVDA · GOOGL · META · TSLA · JPM · V · MA · AMZN
```

n8n Local Setup (Optional)

```
# Start n8n via Docker
docker run -it --rm \
  --name n8n \
  -p 5678:5678 \
  -v ~/.n8n:/home/node/.n8n \
  n8nio/n8n

# Access n8n at: http://localhost:5678
# Import workflow from: n8n/workflow.json
```

Test API Health

```
# Check backend is running
curl http://localhost:8000/latest-report

# Trigger a mock analysis run
curl -X POST http://localhost:8000/run-analysis \
  -H "Content-Type: application/json" \
  -d '{"tickers": ["AAPL"], "no_post": true}'
```

12 n8n Webhook Workflows

n8n is an open-source automation platform (self-hosted via Docker). InvestoraAI posts to 4 n8n webhook endpoints for downstream automation, Telegram notifications, and monitoring.

> ℹ️ n8n webhooks are **optional**. The backend soft-fails gracefully if webhook URLs are blank or unreachable. All core analysis and report generation works without n8n.

Webhook Endpoints

ENV VAR	DEFAULT PATH	TRIGGERED BY	PAYLOAD
`N8N_WEBHOOK_URL`	`/webhook/langgraph-results`	`post_to_n8n` node	Full ReportPayload JSON
`ALERT_WEBHOOK_URL`	`/webhook/investora-alerts`	`post_alerts` node	ALERT_EVENT signals (score ≥8.5 or ≤1.5)
`CANDIDATE_WEBHOOK_URL`	`/webhook/investora-candidates`	`post_candidates` node	WEEKLY_CANDIDATE signals (7.0–8.4)
`MONITOR_WEBHOOK_URL`	`/webhook/investora-monitor`	`persist_snapshot` node	Run metadata (status, error_count, duration)

Signal Classification

SIGNAL TYPE	SCORE RANGE	WEBHOOK	MEANING
ALERT_EVENT	≥ 8.5 or ≤ 1.5	ALERT_WEBHOOK_URL	Strong buy or strong sell signal
WEEKLY_CANDIDATE	7.0 – 8.4	CANDIDATE_WEBHOOK_URL	Watch list candidates for the week
MONITOR	All others	MONITOR_WEBHOOK_URL	Regular tracking / informational

n8n on Production (Render)

⚠️ n8n runs on `localhost:5678` locally. Render's cloud servers cannot reach your local machine, so n8n webhooks will always fail on Render unless you deploy n8n to a cloud service or use n8n Cloud (`https://n8n.io`).

Fix: Deploy n8n to Render as a separate service, or use n8n Cloud, and update the webhook URLs in Render's environment variables.

13 Known Limitations & Future Roadmap

Current Limitations

AREA	LIMITATION	WORKAROUND
Marketstack API	Free tier: 100 req/month only. Full run = ~10–20 requests. Easy to exhaust during testing.	Upgrade to Starter ($9.99/mo) or use `USE_MOCK_DATA=true`
Render Free Tier	Service sleeps after 15 min inactivity. Cold start = 30–60 sec delay.	Upgrade to Starter ($7/mo) for always-on
Ephemeral Storage	Generated reports & SQLite DB lost on Render restart/deploy.	Add Render Disk ($0.25/GB/mo) or commit reports to git
n8n Webhooks	n8n runs locally — not reachable from Render cloud.	Deploy n8n to cloud or use n8n Cloud
No Authentication	Dashboard has no login — anyone with the URL can view and trigger runs.	Planned: Multi-user auth via n8n static data storage
Sentry Backend	sentry-sdk was removed from venv to prevent LangChain crash on Python 3.14.	Re-add after testing; use Python 3.12 (which is already pinned)

Future Roadmap

🔐 Multi-User Authentication

Username/password login with user profiles (risk tolerance, interests, Telegram chat ID) stored in n8n static data. Personalization badges on dashboard.

☁️ Cloud n8n Deployment

Deploy n8n as a Render service or use n8n Cloud to enable Telegram alerts and webhook automations in production.

📊 Sentry Error Tracking

Re-enable Sentry backend monitoring on Python 3.12. Frontend Sentry is already configured via SENTRY_DSN env var.

🗄️ Persistent Database

Add Render Persistent Disk or migrate SQLite to PostgreSQL (Render's managed DB) for reliable run history across deploys.

🔔 Telegram Notifications

Enable per-user Telegram alerts for ALERT_EVENT signals using the user's stored telegramChatId via the n8n workflow.

⏰ Scheduled Weekly Runs

Configure a Render Cron Job or n8n schedule to auto-trigger the weekly analysis every Monday morning without manual intervention.

Quick Reference — Production URLs

SERVICE	URL	STATUS
Frontend Dashboard	`https://investora-ai.vercel.app`	✅ Live
Backend API Root	`https://investoraai.onrender.com`	✅ Live
Latest Report API	`https://investoraai.onrender.com/latest-report`	✅ Live
Run History API	`https://investoraai.onrender.com/run-history`	✅ Live
n8n Local	`http://localhost:5678`	⚠️ Local only

InvestoraAI — AI-Powered Weekly Stock Market Intelligence Platform

Generated: March 2026 · Version 1.0.0 · Built with React + FastAPI + LangGraph